

TeamSync AI

Explainable, Graph-Grounded Multi-Agent
Project Intelligence

Core idea	Compute coordination risk deterministically from a knowledge graph; let the LLM only <i>narrate</i> one finding from a bounded slice of that graph.
Main claim	Risk detection can be made both cheap and trustworthy without losing accuracy , and it can warn while the sprint can still be changed.
Token reduction	487x vs naive prompting (189,834 to 390 tokens at 2,003 tasks)
Mid-sprint warning	AUC 0.792 halfway through a sprint, 0.874 at three-quarters, on 22 unseen projects
Held-out F1	0.712 (zero-parameter graph rule) at sprint end, where a tuned model's cross-validated 0.816 fell to 0.593, on 22 unseen projects
Accuracy ablation	1.000 graph vs 0.947 naive full-dataset prompting, identical scenarios
Evidence grounding	200/200 fabricated citations mechanically stripped
LLM agreement	Cohen kappa 0.95 (n=37) between LLM narration and the deterministic verdict
Real dataset	TAWOS - 458,232 real Jira issues, 39 open-source projects (MSR 2022, Apache 2.0)
Stack	Next.js FastAPI x2 PostgreSQL NetworkX Groq LLM (optional)

Every value in this report is read from the committed evaluation result files at build time. Nothing is hand-transcribed, so the report cannot drift from the experiments that produced it. All results were independently re-audited (Section 8.9).

1. Problem Statement

Teams using conventional project-management tools (Jira, Linear, Asana, Monday.com) run into four compounding failures:

- **Risk is detected late.** Tools show current state, not risk trajectory. An overloaded engineer or a slipping critical-path task becomes visible only after the damage is done.
- **Warnings are opaque.** When a tool flags something as at risk, it rarely shows *which evidence* produced that verdict, so a manager can neither act on it nor challenge it.
- **Signals are siloed.** No single view reveals a cross-cutting risk such as: the only person who understands this component is also unresponsive and on the critical path.
- **Naive AI fixes create new problems.** Feeding raw project data to an LLM is expensive (cost scales with project size), non-deterministic (same input, different verdicts), and produces justifications nobody can verify.

Research question. *Can a system detect coordination risk early, explain it with verifiable evidence, and stay cheap enough to run continuously - without trading away accuracy for that cheapness?*

2. Proposed Solution

Six steps. The inversion in steps 2 to 4 is the entire idea.

- Compile project state into a **typed knowledge graph** (people, tasks, milestones, dependencies, comments) from data the product already stores.
- Compute all six risks as **deterministic graph algorithms** - zero LLM tokens, zero randomness.
- For each detected anomaly, extract a small **witness subgraph** holding only the relevant nodes.
- Give the LLM **only that subgraph**, and ask it to *narrate* - never to decide.
- **Mechanically verify** every citation against the subgraph; discard anything that does not resolve to a real node.
- Every agent has a **rule-based fallback** with an identical schema, so the system works with no LLM at all.

The LLM never sees or judges the raw dataset. It only explains an already-computed, already-verified fact.

Problem	How the solution addresses it
Late detection	Graph algorithms are cheap enough to run on every change, not as a periodic report
Opaque warnings	Every finding carries evidence that is mechanically checkable
Siloed signals	One graph unifies task, people, dependency and comment data
Expensive / unreliable AI	The prompt is bounded; the numeric answer never depends on the LLM

3. Novelty - Four Testable Claims

Each claim is independently testable, and each was tested. Section 8 reports the measurement behind every one.

3.1 Graph-Grounded Agent Prompting

Project state becomes a typed knowledge graph, and risk is computed as **graph invariants** (critical path, articulation points, weighted degree, community structure) rather than inferred by a language model. The LLM receives one finding plus its minimal witness subgraph and must cite node ids drawn from it. This inverts the standard pattern in which an LLM is handed raw data and asked to find problems, and it is what makes the remaining three claims achievable at all.

3.2 Prompt cost is O(anomalies), not O(project size)

Because the LLM only ever sees a bounded subgraph, prompt size stops tracking project size. **Measured: 266 to 390 tokens (1.47x) while the project grows 154x** (13 to 2,003 tasks). Against naive full-dataset prompting at the same size: 189,834 to 390 tokens, a **487x reduction**. This is what makes continuous, per-change analysis economically viable rather than a periodic expensive report.

3.3 Hallucinated evidence is mechanically detectable

Most explainable-AI systems ask an LLM to cite sources and hope. Here every citation is checked against a bounded set of real node ids, and anything unresolvable is stripped after generation. **Measured: 200/200 fabricated references removed (100%).** This converts *the model was told to cite evidence* into *every surviving citation is verifiably real*.

3.4 Added model complexity did not survive unseen projects

Three attempts to improve accuracy with more sophisticated methods **all failed or backfired** on held-out projects (Section 7.5). A tuned logistic model that reached cross-validated F1 0.816 fell to **0.593** on 22 projects it had not seen, where the untuned deterministic rule scored **0.712**. A paired test cannot separate them (exact McNemar $p = 0.78$; bootstrap F1 difference +0.119, 95% CI [-0.025, +0.291]), so the claim is **not** that the rule predicts better. What is significant: the rule beats flagging every sprint by +0.055 F1 (95% CI [+0.029, +0.080], $p < 0.001$), and the learned model's tuned advantage vanished across project boundaries. Complexity bought no measurable accuracy here, and it cost the explanation.

3.5 The warning arrives while the sprint can still be changed

Projected forward from the work completed so far, the same graph flags sprints that will finish late **halfway through** rather than after the deadline. On 270 held-out sprints from 22 unseen projects: **AUC 0.792** at the 50% checkpoint and **0.874** at 75%, with F1 0.706 and 0.756 against 0.657 for flagging every sprint (+0.050, 95% CI [+0.006, +0.094]; +0.099, [+0.049, +0.148]; both $p < 0.001$). Stated honestly, the projection ranks *no better* than the share of work still open (0.802 and 0.881) - its contribution is that the graph explaining a risk also carries it forward in time, with the evidence attached.

Novelty statement for the abstract. Risk detection can be made both cheap (graph-computed, $O(\text{anomalies})$ prompt cost) and trustworthy (mechanically grounded evidence) without sacrificing accuracy to a more complex learned model.

4. Architecture

4.1 Service topology

Three services; the AI service is independently deployable and degrades to a rule-based path without the LLM.

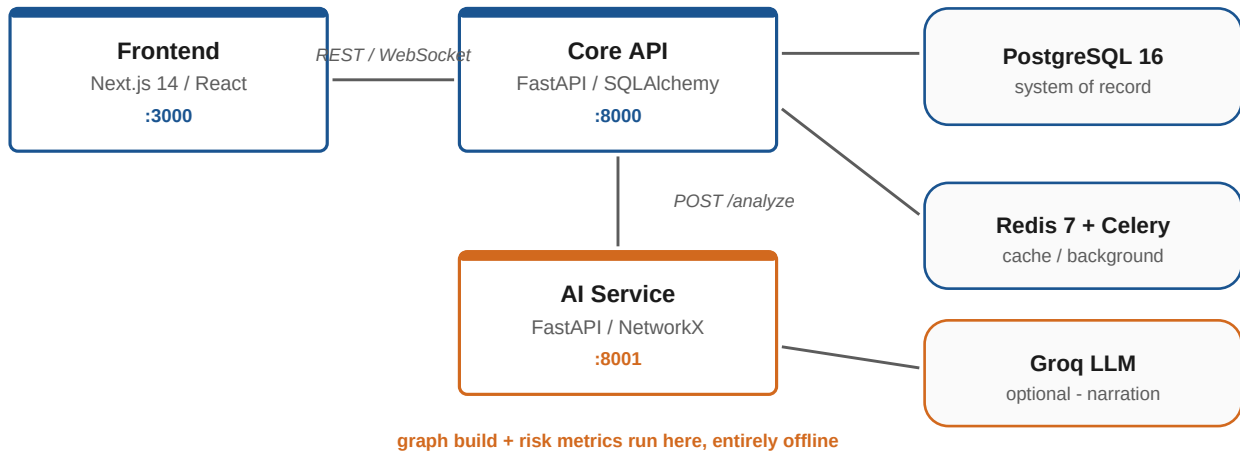


Fig. 1. Service topology. Three independently deployable services. The AI service holds the graph engine and degrades to a rule-based path when no LLM is available.

Service	Technology	Port	Role
Frontend	Next.js 14, React 18, Tailwind, TypeScript	3000	UI, Kanban board, dashboards, WebSocket client
Core API	FastAPI, SQLAlchemy 2.0 async, Pydantic v2	8000	Auth, CRUD, multi-tenancy, persistence
AI Service	FastAPI, NetworkX, Groq client	8001	Graph build, risk metrics, agent pipeline
Database	PostgreSQL 16 (SQLite in tests)	5432	System of record
Cache / Queue	Redis 7 + Celery	6379	Caching, background tasks

4.2 The analysis pipeline - where the contribution lives

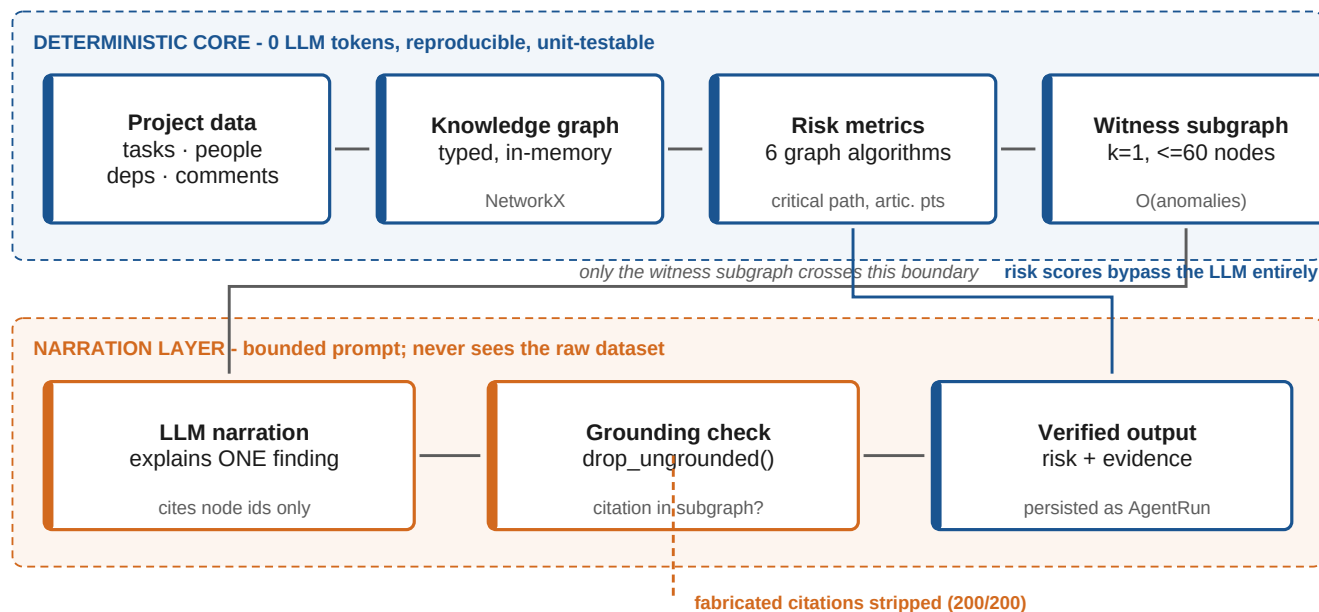


Fig. 2. Graph-grounded agent prompting. All six risks are computed as deterministic graph invariants at zero LLM cost; only the bounded witness subgraph of an anomaly crosses into the narration layer. Risk scores never depend on the language model, and every citation is checked before it reaches the caller.

Steps 1 to 4 and the risk scores involve **no LLM call at all**. Only narration touches the model, and only on a bounded slice of the graph. This is why the numeric output is fully reproducible: identical input always yields identical risk scores, and only the prose varies.

4.3 Knowledge graph schema

Nodes: **Person, Task, Milestone, Project, Component**. Every edge is derived from tables the product already stores, so the graph requires **no new data collection**.

Edge	Derived from
Person -[ASSIGNED_TO {points}]-> Task	task assignee + story points
Person -[REPORTED]-> Task	task reporter
Task -[BLOCKS]-> Task	task dependency
Task -[SUBTASK_OF]-> Task	parent task
Task -[PART_OF]-> Milestone	milestone link
Person -[COMMENTED_ON]-> Task	task comments
Person -[MEMBER_OF]-> Project	project membership
Task -[TOUCHES]-> Component	milestone-as-component proxy
Person -[KNOWS {depth}]-> Component	composition of ASSIGNED_TO and TOUCHES

4.4 The six risks as graph algorithms

Risk type	Graph computation	Threshold
Dependency (critical path)	Longest path through the BLOCKS DAG	CRITICAL_PATH_SHARE $\geq$ 0.25
Dependency concentration	Betweenness centrality of task nodes	-
Knowledge / SPOF	Articulation points in the Person-Component projection	SPOF_MIN_COMPONENT_TASKS = 2
Workload	Weighted degree over ASSIGNED_TO (weight = story points)	WORKLOAD_SKEW $\geq$ 1.5
Coordination	Community structure / isolated members	-
Silent member	COMMENTED_ON degree divided by ASSIGNED_TO degree	SILENT_RATIO $\geq$ 0.30
Delay	Open tasks past their due date	OVERDUE_RATIO $\geq$ 0.10

A correctness win came free with the graph. The pre-existing dependency API allowed creating a three-hop circular dependency (A blocks B blocks C blocks A) because it only checked for direct self-loops. Once the graph exists, `nx.simple_cycles` detects real multi-hop cycles.

4.5 Witness subgraph - the efficiency mechanism

Parameter	Value	Why it matters
DEFAULT_HOPS	1	Only the immediate neighbourhood of an anomaly is included
MAX_NEIGHBORS_PER_SEED	8	Critical. Without it, a high-degree node re-introduces size scaling (measured 4.31x growth before the cap, 1.007x after)
MAX_WITNESS_NODES	60	Hard ceiling; seed nodes always survive truncation

Serialization is **columnar rather than JSON**: field names appear once per block header, for example `tasks(id, status, pts, due, title):`, instead of being repeated on every row. After subgraph selection this accounts for most of the remaining saving.

4.6 Agent pipeline

Coordinator to three specialists in parallel, then a risk aggregator, then a recommendation generator.

Agent	Reads	Produces	Status
Planning	Milestones, capacity, graph metrics	Sprint readiness, capacity gaps	Active
Progress	Tasks, velocity, graph metrics	Velocity trend, stalled work	Active
Workload	Assignments, graph metrics	Overload, single points of failure	Active
Risk	The six graph-computed scores	Aggregated scores plus evidence	Active
Recommendation	Risk plus specialist outputs	Prioritized actions	Active
Meeting Intel	Transcripts	Decisions, action items	Deferred
Comm Intel	Chat / email events	Response delays, participation gaps	Deferred
Review Trio	A proposal	Consensus verdict	Deferred

The three deferred agents are **architecturally complete but have no data source** in the product yet - a documented scope boundary, not a hidden gap. Every agent, LLM-backed or fallback, returns one shared schema: `summary`, `risk_level`, `confidence`, `signals`, `evidence`, `recommendations`, `next_action`, and `metadata` (where a `fallback` flag marks the deterministic path).

5. Features

Area	Capabilities
Product	Organization multi-tenancy; JWT auth with refresh rotation; projects with milestones and health/risk scores; Kanban board with drag-and-drop, dependencies, subtasks and comments; status workflow Backlog to Planned to In Progress to Blocked to Review to Done; WebSocket real-time updates
AI layer	Five active agents covering six risk types; deterministic zero-token risk scoring; optional LLM narration; rule-based fallback for every agent (works with no API key); mechanically enforced evidence grounding; full audit trail persisting specialist outputs, risk scores and the witness subgraph used; real multi-hop cycle detection
Deferred	Meeting Intelligence; Communication Intelligence; Review Trio; Organizational Memory and RAG; Team Intelligence Index; per-project roles; persistent graph storage (Neo4j)

6. Datasets

6.1 TAWOS - the real-world dataset

Property	Value
Full name	A Versatile Dataset of Agile Open Source Software Projects
Citation	Tawosi, Al-Subaihini, Moussa and Sarro, MSR 2022 , doi:10.1145/3524842.3528029
License	Apache 2.0 (citation required)
Content	Real Jira issue-tracker data from 12 public Jira repositories
Size	4.3 GB MySQL dump, parsed with 0 parse errors

Table	Rows	Table	Rows
project	39	issue	458,232
user	206,162	issue_link	246,587
sprint	4,594	comment	1,518,327

Row counts were reconciled exactly against the published totals after loading.

How it is used - sprint-level delay prediction. TAWOS has no `due_date` field, so the label is derived from data it does have: a sprint is *delayed* when at least 30% of its issues were unresolved at sprint end.

Point-in-time reconstruction. Every issue in an archival dataset is long since resolved, so current status cannot be used directly - that would leak the future into a retrospective label. Each task's state is rebuilt *as of sprint end*, and metrics are evaluated with `now = sprint_end + 1 day`.

Eligible sprints (CLOSED, has an end date, at least 15 issues): **987 sprints, 31 projects, 58.8% positive**.

Split discipline. The split is by *project*, not by sprint, so a team's Jira conventions cannot leak across the boundary. The held-out set was evaluated **exactly once**.

Split	Projects	Sprints	Positive rate
Tuning	9	717	-
Held-out	22	270	48.9%

6.2 Synthetic scenarios

180 generated scenarios (90 positive / 90 negative, 30 per risk type) with a **deterministically injected** anomaly, so ground truth is known by construction. Used for implementation-correctness checks, threshold boundary sweeps, and the architecture ablation.

6.3 Datasets considered but not used

Dataset	Intended for	Status
AMI Meeting Corpus	Meeting Intelligence	Deferred - agent not built
Enron Email Corpus	Communication Intelligence	Deferred - agent not built

7. Evaluation - All Measured Values

7.1 Engineering baseline

Item	Value
Backend tests	63 / 63 pass
AI service tests	67 / 67 pass (pytest -m "not deferred")
Frontend tests	50 / 50 pass (vitest), tsc clean
Continuous integration	all three suites and the type check run on every push
Code size	approx. 21,000 lines
Quality gates	ruff, mypy, eslint, prettier, tsc

7.2 Synthetic detection correctness

180 scenarios with the anomaly injected deterministically, so the label is known by construction.

TP	FP	FN	TN	Precision	Recall	F1	Accuracy
90	0	0	90	1.00	1.00	1.00	1.00

State this honestly in the paper. For deterministic threshold code on unambiguous inputs, 100% is the expected result of correctness - equivalent to saying all unit tests pass. It is **not** evidence of generalization, and should not be presented as such. Stability was confirmed across 4 further seeds (1,200 additional trials).

The genuinely falsifiable check is the **boundary sweep**: does each detector flip exactly at its documented threshold?

Signal	Documented threshold	Observed transition
overdue_ratio	0.10	flips at exactly 0.10
silent_ratio	0.30	0.20 to 0.30 (exact)
workload_skew	1.50	bracketed 1.429 to 1.522

7.3 Efficiency - the headline result

Full two-series sweep, identical measurement method for both architectures.

Project size (tasks)	Graph-grounded	Naive full-dataset	Ratio
13	266	1,050	3.9x
53	386	4,695	12.2x
203	389	18,901	48.6x
1,003	389	94,724	243.5x
2,003	390	189,834	486.8x

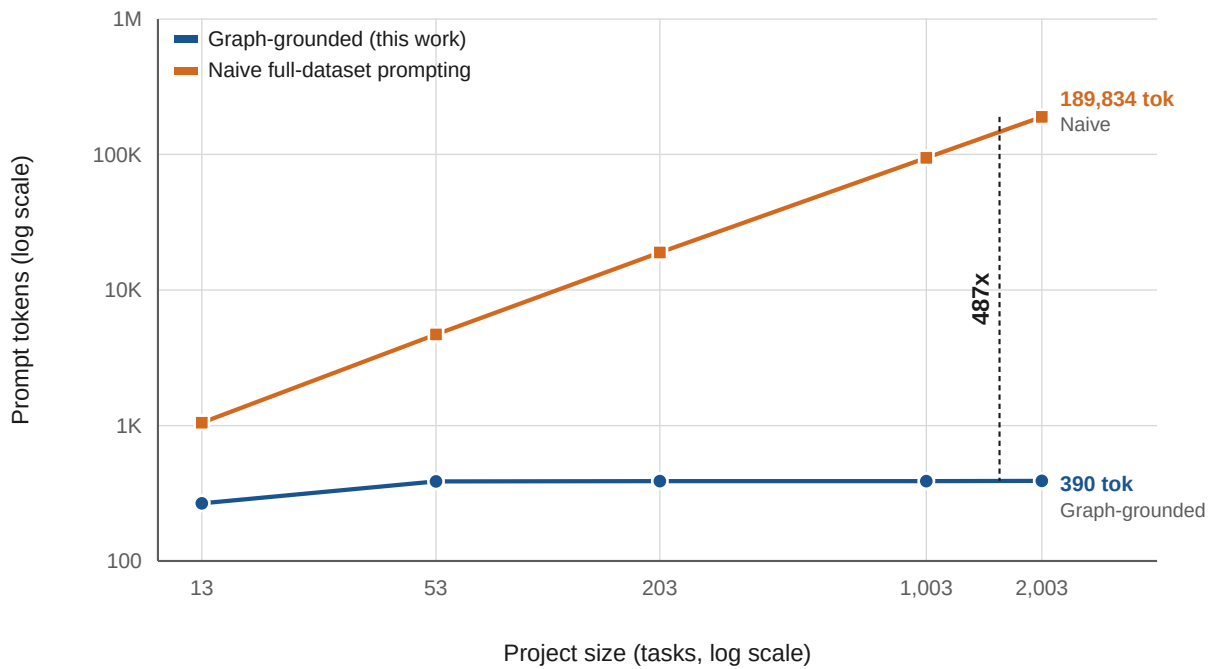


Fig. 3. Prompt cost against project size, both axes logarithmic. Witness-subgraph selection holds prompt size effectively constant while the project grows 154x, whereas naive prompting grows linearly to 189,834 tokens.

Derived metric	Value
Token growth vs project-size growth	1.47x tokens for 154x project size
Reduction at 2,003 tasks	99.8% (487x)

Latency of the deterministic path (graph build + metrics + subgraph selection; 203 tasks, n=50):

p50	p95	mean	max
10.96 ms	13.42 ms	11.87 ms	50.58 ms

Latency varies with machine load between runs (p50 observed between 4.8 and 11.0 ms across runs). The defensible claim is single- to low-double-digit milliseconds, which is what justifies running analysis **synchronously**, without a queue.

Evidence grounding enforcement: 200/200 fabricated references stripped (100%).

7.4 Real-world results - TAWOS

All 987 sprints (default, untuned parameters):

Metric	GraphMetrics	Story-point baseline
Precision	0.630	0.545
Recall	1.000	0.386
F1	0.773	0.452
Accuracy	0.654	0.450
AUC-ROC	0.581	-
Brier score	0.345	-

Mid-sprint result - 270 held-out sprints, 22 unseen projects. This is the headline number.

Each sprint is rebuilt as it stood at a checkpoint - only issues created by then exist, only those resolved by then are done - and the system projects the share of scope still unfinished at the deadline. Checkpoints, prediction and severity cut-offs were fixed before the held-out projects were touched; nothing was tuned.

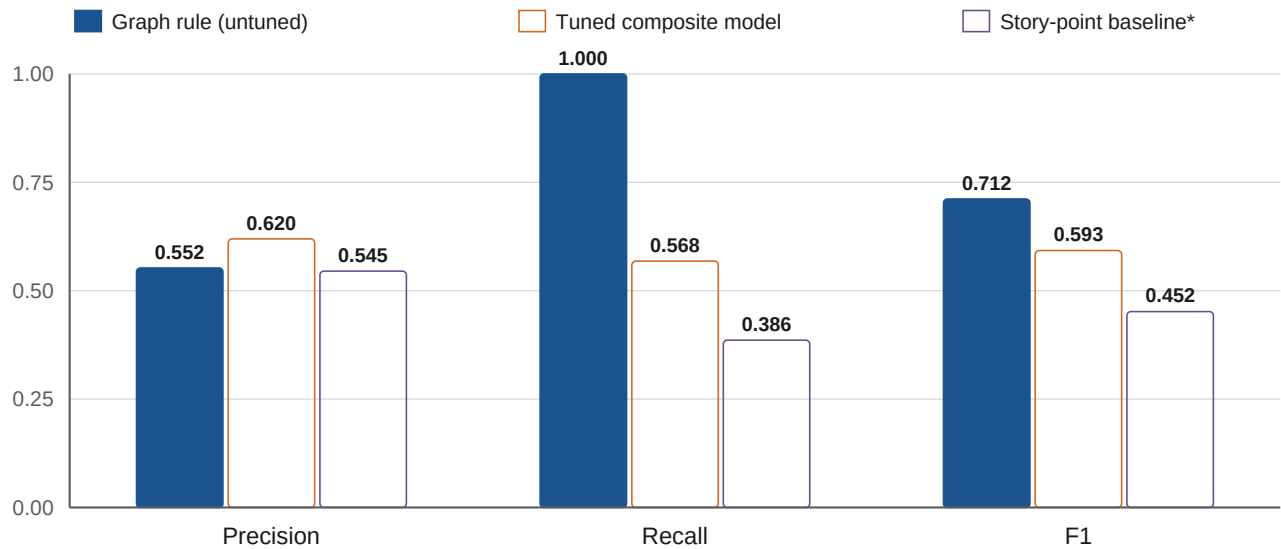
Checkpoint	Predictor	Precision	Recall	F1	AUC
50%	Delay risk not low	0.581	0.901	0.706	0.792
50%	Flag every sprint	0.489	1.000	0.657	-
75%	Delay risk not low	0.623	0.962	0.756	0.874
75%	Delay risk high or worse	0.719	0.909	0.803	-
75%	Flag every sprint	0.489	1.000	0.657	-

Interpretation. Halfway through a sprint the warning is real: it drops a quarter of what flagging everything would flag and still catches 90% of the sprints that finish late (+0.050 F1, 95% CI [+0.006, +0.094]; at 75%, +0.099, [+0.049, +0.148]; both $p < 0.001$). **What it does not show:** better ranking than a trivial baseline - the share of work still open ranks these sprints just as well (AUC 0.802 and 0.881).

Sprint-end result - 270 sprints, 22 unseen projects. Sanity check on the same rule after the deadline.

Model	Precision	Recall	F1	Accuracy	AUC
Graph rule (untuned, 0 parameters)	0.552	1.000	0.712	0.604	-
Tuned composite logistic model	0.620	0.568	0.593	0.619	0.657

Confusion matrices - graph rule: TP 132, FP 107, FN 0, TN 31. Composite model: TP 75, FP 46, FN 57, TN 92.



*baseline measured on all 987 sprints; the other two on the 270-sprint held-out split

Fig. 4. Sprint-delay prediction on the held-out TAWOS split. The zero-parameter graph rule scores alongside a composite logistic model tuned on the other nine projects; a paired test cannot separate them. The story-point baseline is measured on all 987 sprints.

Interpretation. The system flags **every** delayed sprint (recall 1.00, $FN = 0$) while over-flagging roughly half the on-time ones (precision 0.552). Read that recall carefully: after the deadline, a delayed sprint is one with work unfinished past the due date, which is exactly the condition the rule tests, so recall 1.00 is **guaranteed by the label definition here** rather than earned. The mid-sprint numbers above are the predictive ones. What this does show is the operating point: a **high-sensitivity early warning**, not a precise classifier - the right trade when a false alarm is cheap to dismiss but a missed one is not.

7.5 Negative results - three failed improvement attempts

Reported prominently rather than buried. They establish that the simple rule is not leaving accuracy on the table, and they are what make the headline claim credible rather than cherry-picked.

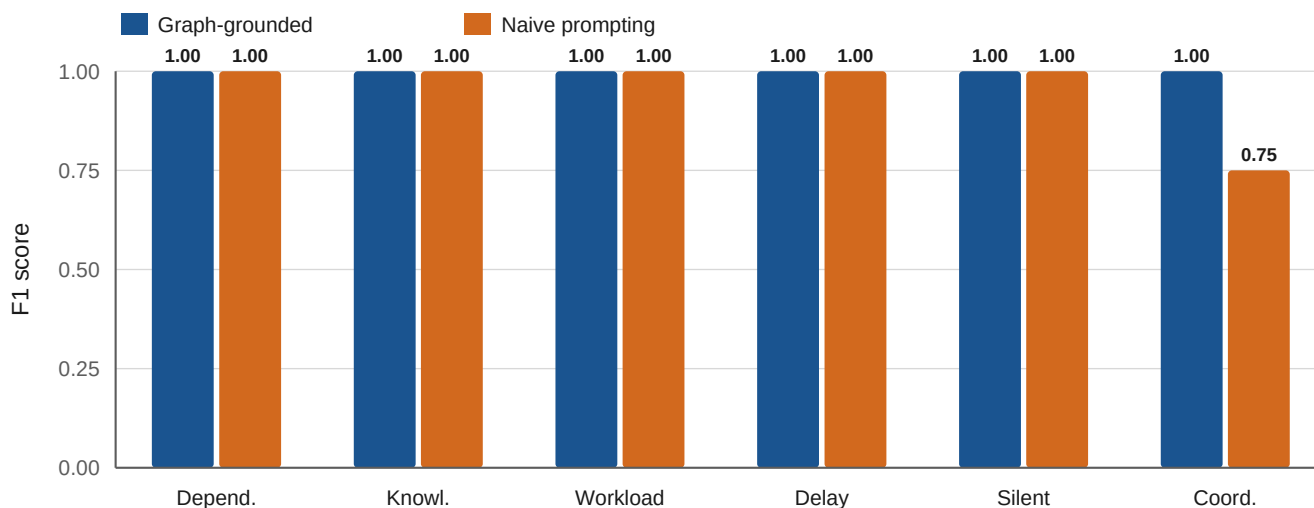
#	Attempt	Outcome
1	Threshold sweep** (0.05 to 0.50)	Provably inert - byte-identical output at every value. With one shared sprint-end due date, overdue_ratio is always exactly 0.0 or 1.0, so no threshold strictly between 0 and 1 can change any classification.
2	Individualized due dates** from tuning-set cycle time (2.96 days per story point)	Un-collapsed the signal (median ratio 0.76, 521/717 sprints intermediate) but lowered AUC from 0.581 to 0.531 . Story points are too noisy a duration proxy.
3	Composite logistic regression**, four features, 3-fold group CV by project	Won on tuning data (CV F1 0.816 , AUC 0.887) and lost on held-out data (F1 0.593) . The dominant weight fell on idle_member_ratio, which is <i>mechanically</i> coupled to the label rather than independently predictive; its calibration is team-specific and did not transfer. The AUC collapse from 0.887 to 0.658 is the signature of overfitting to project-specific structure.

Conclusion. The tuned model's advantage did not cross the project boundary: CV F1 0.816 fell to 0.593, and a paired test cannot separate it from the zero-parameter rule (exact McNemar $p = 0.78$). Added complexity bought no measurable accuracy here, and it cost the explanation: the rule's verdict arrives with the graph nodes behind it, the logistic model's does not.

7.6 Ablation - does graph-grounding cost accuracy?

Both architectures scored on **identical** scenarios against **identical** injected ground truth (n=36, 18 positive / 18 negative, six per risk type; **0 failed calls**, so the comparison is strictly like-for-like).

Architecture	Precision	Recall	F1	Accuracy
Graph-grounded (0 tokens, deterministic)	1.000	1.000	1.000	1.000
Naive full-dataset prompting	0.900	1.000	0.947	0.944



overall F1: graph 1.000 vs naive 0.947 (n=36 scenarios, 0 failed calls)

Fig. 5. Detection accuracy of the graph architecture against naive full-dataset prompting. The two are indistinguishable on five of six risk types; the entire gap is coordination.

Risk type	Graph F1	Naive F1	Naive precision
dependency	1.00	1.00	1.00
knowledge	1.00	1.00	1.00
workload	1.00	1.00	1.00
delay	1.00	1.00	1.00
silent_member	1.00	1.00	1.00
coordination	1.00	0.75	0.60

The design deliberately favours the naive path, so a graph win cannot be dismissed as a strawman: small projects (10 to 20 tasks, its best case), full context delivered in one call, a good-faith prompt defining all six risk types, and lenient response parsing.

What this does not show - three honest qualifications:

- **The naive baseline is not bad.** F1 0.947 is respectable. It reports 2.75 risk types per scenario against the graph's 2.42, with only **12.1%** of its claims uncorroborated by the deterministic graph. It over-reports, but only modestly.
- **Scoring is per-target-risk-type**, so extra risks reported on other dimensions are not counted against it. Stricter all-dimension scoring would likely widen the gap, but ground truth for the non-target dimensions is not established by construction, so it is not claimed.
- **Nothing is shown about naive accuracy at scale.** This tests 10 to 20 task projects by design. Whether the naive path holds F1 near 0.95 at 2,000 tasks is untested, and largely untestable on any reasonable budget - which is itself part of the argument.

Token note. Mean prompt size in this ablation was 1109 (graph) against 1804 (naive) - only 1.6x, and **not** the headline efficiency result. These are deliberately tiny projects. The 487x gap appears at 2,003 tasks. The two numbers must not be conflated.

7.7 Live-LLM metrics - the narration layer

Three runs were made; the third is reported. Model: openai/gpt-oss-120b (Grok deprecated the Llama models originally targeted).

Run	Scenarios	Calls	max_tokens	Pacing	Schema validity	Kappa (n)
v1	18	54	600	4 s	27.8%	0.35 (15)
v2	18	54	2000	18 s	61.1%	0.84 (33)
v3 (reported)	30	90	2000	20 s + backoff	41.1%	0.95 (37)

Cohen kappa = 0.9533 (n=37) - almost perfect agreement on the Landis-Koch scale between the LLM's narrated risk_level and the deterministic fallback's verdict on identical input. This is direct evidence for the central design claim: **the LLM narrates rather than re-decides**.

Risk type	n	Kappa
dependency	14	1.00
knowledge	12	0.86
workload	8	1.00
coordination	1	1.00
delay	1	1.00
silent_member	1	1.00

Schema validity needs two numbers, not one. Of 24 diagnosed failures, **22 were HTTP 429 rate-limits** from the free-tier API key - an infrastructure constraint of the account used, not a property of the model or the pipeline.

Definition	Value
Over all attempted calls (including rate-limit blocks)	41.1% (37/90)
Over calls that actually reached the model	54 to 95% (bounded)

Only the first number is measured. The second cannot be: the run diagnosed 24 of the 50 failures individually and not the rest, so validity over calls that reached the model is $37/68 = 54\%$ if every undiagnosed failure reached it and $37/39 = 95\%$ if none did. The often-quoted 92% assumes the optimistic extreme. Quote the first as the end-to-end rate on free-tier infrastructure and the range for the model itself, until the run is repeated with per-call error logging.

7.8 Metric glossary

Metric	What it measures
Precision / Recall / F1	Classification quality of the risk detector
Accuracy	Overall correctness (less informative under the 59/41 class imbalance)
AUC-ROC	Threshold-independent rank quality of the risk score
Brier score	Calibration quality of the emitted confidence
Point-biserial correlation	Feature screening before building the composite model
Cohen kappa	Agreement between LLM narration and the deterministic verdict
Token count	Direct evidence for the O(anomalies) claim
Latency p50 / p95	Justifies synchronous, non-queued execution
Grounding rate	Direct evidence for the mechanical-verification claim

7.9 Independent verification

Every number was re-audited from scratch rather than re-quoted:

- Confusion-matrix arithmetic recomputed by hand for both models - exact match.

- Zero project overlap between splits confirmed; $717 + 270 = 987$ partitions cleanly with no gaps and no case or whitespace bugs.
- The composite model's normalization statistics confirmed frozen from tuning-only fitting and correctly applied, not silently recomputed, on the held-out set.
- Five randomly sampled held-out sprints' ground-truth labels cross-checked against **raw SQL**, bypassing the ORM entirely - exact match on every sprint.
- The AUC implementation verified against a brute-force $O(n^2)$ pairwise comparison plus three known-answer cases - exact match to full float precision.
- The Brier implementation verified against three known-answer edge cases - exact match.
- Both headline scripts re-run fresh from a clean state - **byte-identical** output.
- SQLite row counts re-queried directly and matched against TAWOS published totals.

No discrepancy was found in any check.

7.10 Engineering rigor - defects found and fixed

Useful for a lessons-learned or methodology-credibility paragraph in the paper.

Defect	Impact
Witness subgraph expanded <i>all</i> neighbours of a seed	Prompt size still scaled with project size (4.31x growth); fixed with a per-seed neighbour cap, reducing growth to 1.007x
Grounding treated an empty findings list as no filter	Silently allowed every citation through; fixed to distinguish empty from absent
Coordinator forwarded the wrong input type to specialists	Every specialist failed on every run , swallowed into an error stub by <code>gather(return_exceptions=True)</code>
Risk aggregation keyed by agent name but looked up by risk type	Risk output was constant regardless of input - the system's central claim produced a fixed answer
Login credentials bound as query parameters	Passwords in access logs and browser history; fixed with form-encoded auth
Due date equal to evaluation time in the TAWOS harness	<code>due < now</code> was never true, giving recall 0.0 on the first full run
Dependency link names guessed rather than inspected	Matched only about 2,000 of roughly 18,600 real links until corrected
A proposed threshold sweep reproduced the label definition	Would have yielded a circular F1 of 1.00; caught and discarded before it reached results

8. Limitations and Threats to Validity

These must appear in the paper. Do not bury them.

#	Threat
1	The delay label is a proxy. TAWOS has no due dates; at least 30% unresolved at sprint end is defensible but not canonical.
2	Only one of six risk types is validated on real data. SPOF, cycles and coordination are near 0% non-zero in sprint reconstruction, because issue links span a project's whole history rather than one sprint window. The other five are validated synthetically only .
3	blocked_ratio never fires - the TAWOS status vocabulary contains no blocked state.
4	AUC is weak by construction at sprint level, since all tasks in a reconstructed sprint share one deadline.
5	The live-LLM sample is thin and imbalanced. Kappa rests on n=37, with three of six risk types at n=1. Single provider, single model, one rate-limited key.
6	No human evaluation of explanation quality. Grounding proves citations are real, not that the explanations are <i>useful</i> .
7	The mid-sprint projection ranks no better than counting open work (AUC 0.792 vs 0.802 at the halfway checkpoint, 0.874 vs 0.881 at three-quarters). Its contribution is the early, evidence-carrying warning, not rank quality.
8	Sprint-end recall of 1.00 is structural , guaranteed by the label definition at that evaluation point; only the mid-sprint recall is predictive.
9	Nothing is claimed about the first quarter of a sprint. The pace signal is deliberately inactive until a quarter of the sprint has elapsed.
10	Live-LLM schema validity is a range (54 to 95%), not a number; only the end-to-end 41.1% is measured.
11	Two designed agents are unimplemented (Meeting, Communication Intelligence) because no data source exists yet.
12	No longitudinal study. There is no evidence yet that a flagged risk predicts a real-world outcome over weeks.

9. Related Work Positioning

Position the contribution against three groups:

Group	Examples	What they lack
Traditional PM tools	Jira, Linear, Asana, Monday.com	Surface current state, not predictive evidence-linked risk; at-risk flags are opaque
LLM multi-agent systems	AutoGPT, MetaGPT, ChatDev, AgentBench	Optimize task completion by LLM agents; no deterministic, mechanically checkable risk computation
XAI in software engineering	SHAP / LIME defect prediction, explainable effort estimation	Explain a <i>learned model's</i> prediction. Here the computation is a deterministic graph invariant needing no post-hoc explanation; the LLM's role is narration under a citation constraint

The differentiating combination - no comparator has all five. Specialist multi-agent decomposition, plus graph-computed (not LLM-inferred) risk, plus mechanically enforced evidence grounding, plus a deterministic fallback with an identical schema, plus a held-out real-world evaluation that reports its negative results.

10. IEEE Paper Structure Mapping

Suggested 8 to 12 pages for IEEE Access; 6 to 8 for a conference paper.

Sec.	Paper section	Source in this report	Pages
-	Abstract	Section 1 summary plus the novelty statement in 3.4	200 words
I	Introduction	Section 1 problem, Section 9 gap, Section 2 approach, Section 3 contributions	1 - 1.5
II	Related Work	Section 9 (build out the three-group table)	1 - 1.5
III	System Architecture	Section 4, with Figs. 1 and 2	1.5 - 2
IV	Graph-Grounded Prompting** (core)	Sections 3.1 to 3.3, 4.3 to 4.5, and 7.10 as evidence of rigor	2 - 3

Sec.	Paper section	Source in this report	Pages
V	Multi-Agent Design	Section 4.6 plus the shared output schema	1 - 1.5
VI	Experimental Setup	Section 6 datasets, split discipline, point-in-time reconstruction	1
VII	Results	Sections 7.2 to 7.7, with Figs. 3, 4 and 5	2 - 3
VIII	Discussion	Section 7.5 negative results, Section 3.4, plus an ethics paragraph	1
IX	Threats to Validity	Section 8 - reproduce all twelve honestly	0.5
X	Conclusion and Future Work	Section 1 plus Section 12	0.5

Tone guidance. The strongest defensible claim is: a simple, explainable, zero-parameter graph rule matches a tuned learned model on unseen projects at 487x lower prompt cost, and warns halfway through a sprint rather than after it. Do not overclaim beyond that. Avoid beats for the model comparison (the paired test cannot separate them) and more accurate for the mid-sprint ranking (a trivial open-work baseline ranks as well). Report the negative results prominently rather than burying them - they are what make the headline credible.

An ethics paragraph is worth writing. Risk-scoring *people* - workload pressure, silent-member detection - carries surveillance and fairness concerns. Acknowledging this directly strengthens the paper rather than weakening it.

11. Reproducibility

Running the system:

Command	Effect
cp .env.example .env	GROQ_API_KEY optional - full fallback mode works without it
make up && make db-seed	Start the full stack and seed demo data (tables are created on startup)
make test	backend 63, ai-service 67, frontend 50 - also run in CI on every push

Reproducing every number in this report (run from `ai-service/`):

Command	Produces
python -m eval.run_eval	Synthetic detection, token sweep, latency, grounding
python -m eval.boundary_eval	Threshold-boundary sensitivity
python -m eval.datasets.tawos_load	TAWOS .sql to SQLite (approx. 4.3 GB, one pass)
python -m eval.datasets.tawos_split	The fixed project-level split
python -m eval.datasets.tawos_score	The all-987-sprint result
python -m eval.datasets.tawos_holdout_eval	The one-shot held-out number
python -m eval.ablation_naive_vs_graph	Naive-vs-graph ablation (needs API key, billed)
python -m eval.live_llm_eval	Schema validity and Cohen kappa (needs API key, billed)
python -m eval.make_figures	Regenerate all five figures (offline)
python -m eval.make_report_pdf	Regenerate this report (offline)

Required citation. Tawosi, V., Al-Subaihin, A., Moussa, R., and Sarro, F. A Versatile Dataset of Agile Open Source Software Projects. MSR 2022. doi:10.1145/3524842.3528029

12. What Is Still Missing

An honest gap list, in priority order.

Priority	Gap	Effort	Why it matters
1	Bootstrap confidence intervals** on the held-out F1	about half a day	Turns a single point estimate into a defensible interval - the most likely reviewer request
2	Bibliography** (15 to 20 references)	1 day	Comparators are currently named but not cited
3	IEEEtran conversion** plus svg-to-pdf	1 day	Submission format
4	Human evaluation of explanation quality	1 to 2 weeks	The biggest unmeasured claim for a paper about explainability
5	Grounding ablation (run with the check disabled)	about 1 day	Shows the safety net catches real hallucinations, not just synthetic fabrications
6	Balanced live-LLM sample on a paid tier	days	Fixes the n=1 categories in the kappa breakdown
7	Validate the remaining five risk types on real data (AMI / Enron)	months	Removes the largest scope limitation

Venue readiness:

Venue	Ready?
Regional IEEE conferences (ICCCNT, ICACCS, ICOEI, ICSSIT)	Yes - comfortably above the typical bar
IEEE Access	Yes, after items 1 to 3
IEEE ICSME / SANER	Borderline - add items 4 and 5
IEEE TSE / ICSE / ASE	No - needs items 4 through 7

License: MIT. Third-party components: Groq, NetworkX, FastAPI, SQLAlchemy, Pydantic, Next.js, shadcn/ui, and the TAWOS dataset (Apache 2.0, MSR 2022 - citation required).